☰ TOC

R3.05 - Thread et synchron

Préambule

Nous avons étudié les processus comme unités d'exécution indépendantes avec leur propre espace mémoire. Les **threads** (ou "fils d'exécution") constituent une évolution de ce concept. Ils ont été développés comme une réponse aux limitations fondamentales des processus traditionnels. Le concept de "processus léger" (lightweight process) a été formalisé par les laboratoires AT&T Bell dans les années 1980, puis standardisé avec POSIX Threads (Pthreads) en 1995. Java, dès sa conception en 1995, a intégré nativement cette notion de threads, permettant le multithreading.

Dans le cour précédent, que lorsqu'un processus crée un nouveau processus (fils) via `fork()` , le système d'exploitation duplique intégralement cette structure mémoire, créant deux espaces d'adressage complètement séparés. Cette séparation garantit l'isolation mais empêche tout partage direct de données et est lent.

- Coût de Création : L'Overhead de fork()
- Duplication du Process Control Block (PCB) : Structure contenant l'état complet du processus
- Copie de l'espace d'adressage : Duplication de toutes les pages mémoire
- Duplication des descripteurs de fichiers : Copie de la table des fichiers ouverts
- Création d'un nouveau PID : Attribution d'un identifiant unique dans l'espace système.
- Initialisation du contexte d'exécution : Configuration des registres et de la pile

Cette séquence peut prendre plusieurs millisecondes, rendant la création fréquente de processus prohibitive pour des applications nécessitant une réactivité élevée. Sans parlé des méchanisme de communication interprocessus.

Les threads révolutionnent cette architecture en introduisant un partage sélectif des ressources. Au lieu de dupliquer l'intégralité de l'espace mémoire, les threads d'un même processus partagent tous les segments à l'exception de la pile d'exécution.

Processus vs Threads

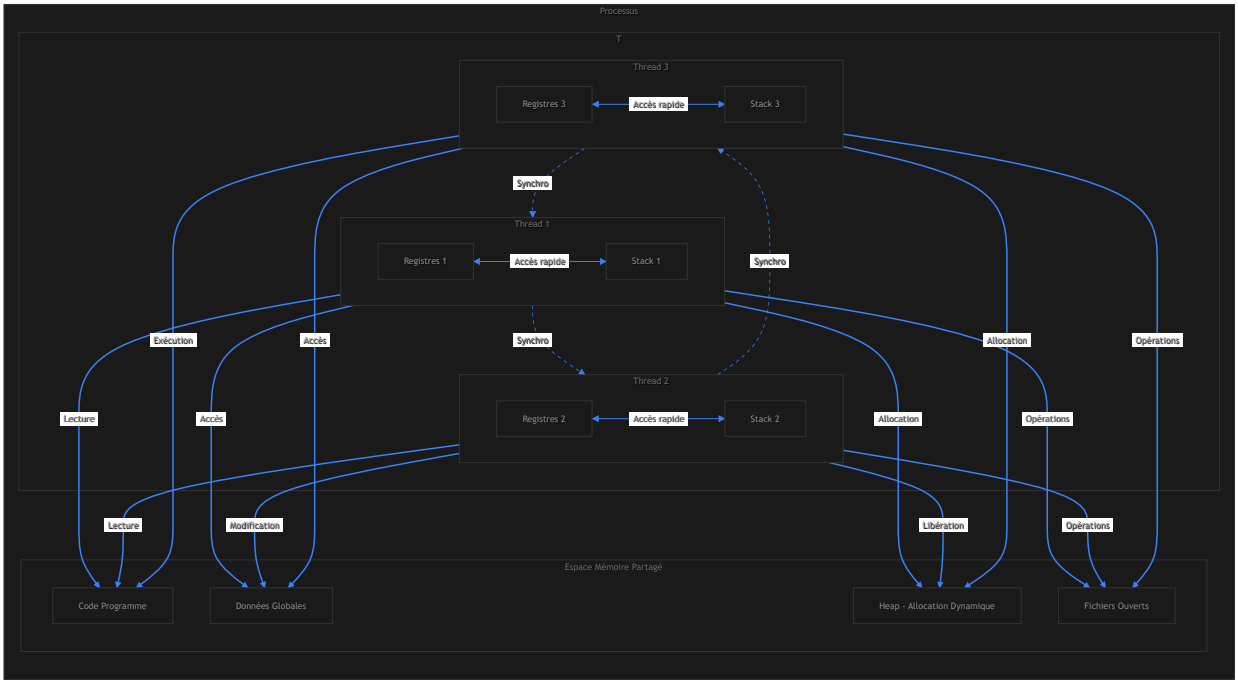
Un thread peut être vu comme un `processus léger` qui s'exécute à l'intérieur d'un processus parent. Contrairement aux processus qui possèdent chacun leur propre espace mémoire, les threads d'un même processus partagent :

- **Le même espace d'adressage** : code, données globales, heap
- **Les mêmes ressources système** : fichiers ouverts, connexions réseau
- **Le même contexte de sécurité** : permissions, utilisateur

Chaque thread possède néanmoins :

- **Sa propre pile (stack)** : variables locales, appels de fonctions
- **Ses propres registres CPU** : compteur de programme, registres de travail
- **Son propre état d'exécution** : en cours, bloqué, prêt

Cette architecture hybride permet une communication optimal (accès direct aux variables partagées) tout en préservant l'indépendance de l'exécution (pile privée pour chaque thread).



Cette architecture permet une communication optimal entre threads (pas de copie de données entre espaces mémoire séparés) mais introduit de nouveaux défis de synchronisation.







Regarder sur

Avantage de l'architecture

Contrairement aux processus qui sont ordonnancés indépendamment par le noyau, les threads d'un processus partagent le même contexte d'exécution tout en maintenant leur propre flot de contrôle. Cette dualité créé des opportunités extraordinaires mais aussi des défis.

Avantages architecturaux :

- Création rapide : Pas de duplication d'espace mémoire, seulement allocation d'une nouvelle pile.
- Communication Directe : Variables partagées accessibles instantanément sans marshalling/unmarshalling
- Économie Mémoire : Code et données partagés entre tous les threads
- Switching Rapide : Changement de contexte minimal (registres + pointeur de pile), donc quelques cycle cpu maximum.

Défis introduits :

- Race Conditions : Accès concurrent non synchronisé aux données partagées
- Deadlocks : Situations d'interblocage mutuel entre threads
- Visibilité mémoire : Incohérences dues aux caches processeurs
- Debugging complexe : Comportements non-déterministes difficiles à reproduire

Attention le kernel peut toujours préempter le processus et/ou les threads

Implémentation en Java

Java intègre nativement le support des threads. Voici les deux approches principales pour créer un thread :

Approche 1 : Hériter de Thread

```
class MonThread extends Thread {
    private String nom;

    public MonThread(String nom) {
        this.nom = nom;
    }

    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println(nom + " - étape " + i);
            try {
                Thread.sleep(1000); // Pause d'1 seconde
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class ExempleThread {
    public static void main(String[] args) {
        MonThread t1 = new MonThread("Thread-1");
```

```
        MonThread t2 = new MonThread("Thread-2");

        t1.start(); // Démarre l'exécution parallèle
        t2.start();

        t1.join(); // Attente de fin d'éxécution
        t2.join();
    }
}
```

Approche 2 : Implémenter Runnable

```
class MaTache implements Runnable {
    private String nom;

    public MaTache(String nom) {
        this.nom = nom;
    }

    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println(nom + " - étape " + i);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class ExempleRunnable {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MaTache("Tâche-1"));
        Thread t2 = new Thread(new MaTache("Tâche-2"));

        t1.start();
        t2.start();

        try {
            t1.join(); // Attend la fin de t1
            t2.join(); // Attend la fin de t2
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Toutes les tâches terminées");
    }
}
```

**Approche 3 : **

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

class MaTache implements Callable<String> {
    private String nom;

    public MaTache(String nom) {
        this.nom = nom;
    }

    @Override
    public String call() throws Exception {
        for (int i = 0; i < 5; i++) {
            System.out.println(nom + " - étape " + i);
            try {
                Thread.sleep(1000); // simule un travail
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
```

```
        }
        return nom + " terminée !";
    }
}

public class ExempleCallable {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        // On soumet deux tâches
        Future<String> f1 = executor.submit(new MaTache("Tâche-1"));
        Future<String> f2 = executor.submit(new MaTache("Tâche-2"));

        try {
            // Récupération des résultats (bloque jusqu'à la fin)
            String resultat1 = f1.get();
            String resultat2 = f2.get();

            System.out.println(resultat1);
            System.out.println(resultat2);

        } catch (InterruptedException | ExecutionException e) {
            e.printStackTrace();
        }

        executor.shutdown(); // libère les ressources
        System.out.println("Toutes les tâches terminées");
    }
}
```

Problèmes de concurrence

L'exécution simultanée de plusieurs threads partageant des ressources communes peut créer des situations problématiques. Les principales difficultés sont les **conditions de course** (race conditions) et les **problèmes de visibilité**.

L'ordre d'éxécution est non-déterministe : on ne sait pas qui seras écécuté et dans quel ordre.

Lecture concurrent (thread-safe)

```
public class LectureConcurrente {
    static int number = 3; // Variable partagée en lecture seule

    static class ThreadLecteur implements Runnable {
        private final String nom;
        private final int operande;

        public ThreadLecteur(String nom, int operande) {
            this.nom = nom;
            this.operande = operande;
        }

        @Override
        public void run() {
            // Variables locales : chaque thread a sa propre pile
            int x = operande;

            // Lecture seule de la variable partagée : THREAD-SAFE
            int resultat = x + number;

            System.out.println(nom + ": " + x + " + " + number + " = " + resultat);
        }
    }

    public static void main(String[] args) throws InterruptedException {
        System.out.println("=== Test de lecture concurrente (Thread-Safe) ===");

        for (int iteration = 1; iteration <= 5; iteration++) {
            System.out.println("\n--- Itération " + iteration + " ---");

            Thread threadA = new Thread(new ThreadLecteur("Thread-A", 5));
            Thread threadB = new Thread(new ThreadLecteur("Thread-B", 2));
```

```
            threadA.start();
            threadB.start();

            threadA.join();
            threadB.join();
        }

        System.out.println("\nRésultat : Toujours A=8, B=5 (déterministe)");
        System.out.println("Seul l'ordre d'affichage peut varier");
    }
}
```

Dans cet exemple, bien que l'ordre d'affichage puisse varier, les valeurs calculées sont toujours identiques car :

- `number` n'est que lu (pas de modification concurrente)
- Les variables `x` sont locales à chaque thread (piles séparées)
- Les calculs sont purement fonctionnels

A partir du moment ou quelqu'un écris : aucune garantie

Race conditions

Les race conditions surviennent lorsque plusieurs threads accèdent simultanément à une ressource partagée et qu'au moins un thread modifie cette ressource. Le terme `race` fait référence à une "course" entre les threads pour accéder à la ressource.

Facteurs aggravant les race conditions :

- Optimisations CPU : Réordonnancement d'instructions, exécution spéculative
- Caches multi-niveaux : L1, L2, L3 pouvant contenir des valeurs différentes
- Architecture multi-cœurs : Chaque cœur peut avoir une vision différente de la mémoire
- Compilateur : Optimisations pouvant réorganiser le code
- JVM : JIT compiler, garbage collector, optimisations runtime

```
class CompteurDangereux {
    private int count = 0;

    public void incrementer() {
        count++; // Opération NON atomique !
    }

    public int getCount() {
        return count;
    }
}

public class ExempleRaceCondition {
    public static void main(String[] args) throws InterruptedException {
        CompteurDangereux compteur = new CompteurDangereux();

        Runnable tache = () -> {
            for (int i = 0; i < 1000; i++) {
                compteur.incrementer();
            }
        };

        Thread t1 = new Thread(tache);
        Thread t2 = new Thread(tache);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        // Résultat attendu : 2000
        // Résultat réel : souvent < 2000 !
        System.out.println("Count final : " + compteur.getCount());
    }
}
```

Ce test révèle généralement :

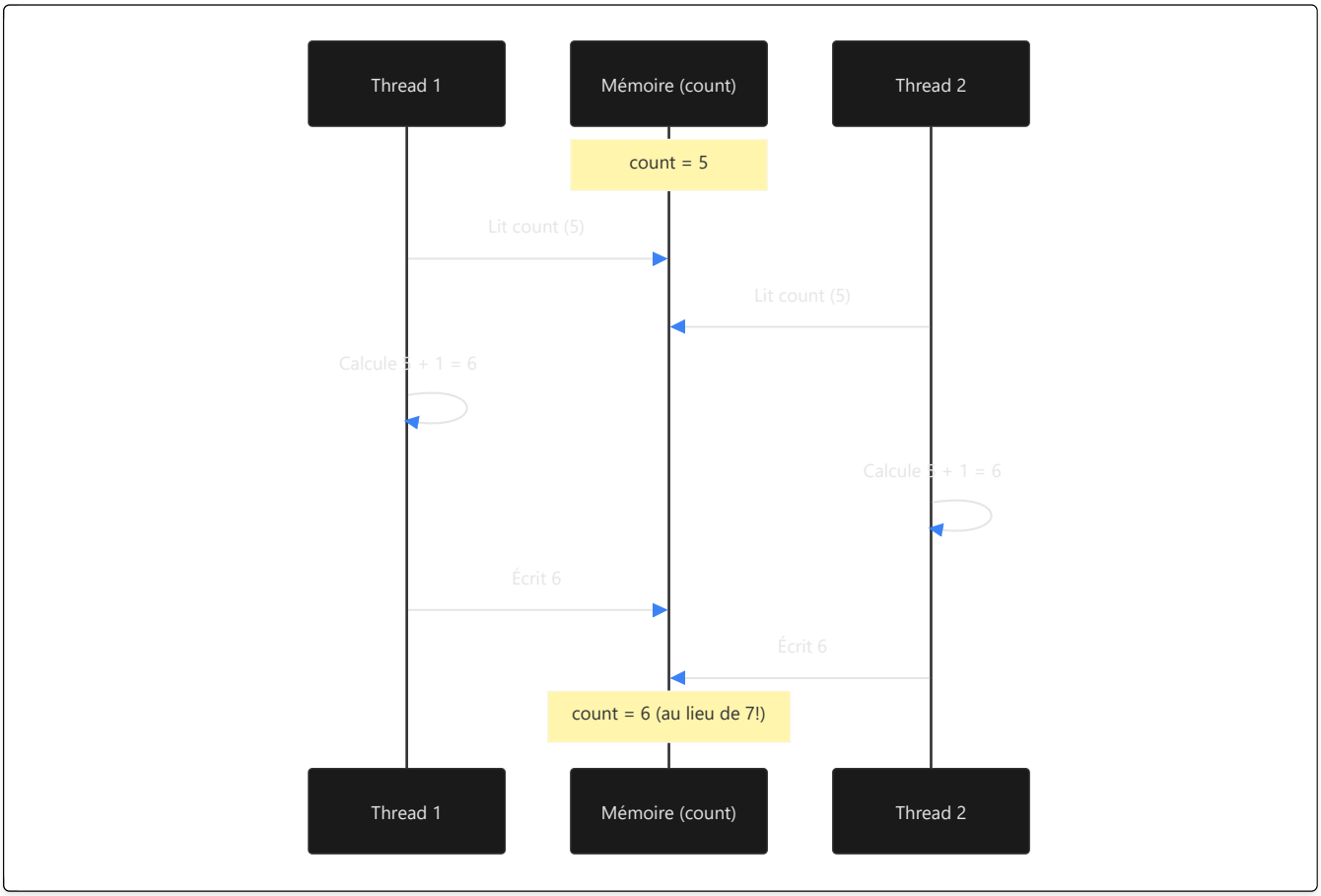
- Version dangereuse : Résultats variant entre 60% et 95% de la valeur attendue
- Version sécurisée : Toujours 100% correct, mais légèrement plus lent
- Variabilité : Les résultats changent à chaque exécution

Pourquoi ce code est-il problématique ? L'opération `count++` semble atomique mais se décompose en réalité en trois étapes :

1. **Lecture** de la valeur actuelle de `count`
2. **Calcul** de la nouvelle valeur (`count + 1`)

3. **Écriture** de la nouvelle valeur dans `count`

Si deux threads exécutent ces étapes simultanément, ils peuvent lire la même valeur initiale, l'incrémenter séparément, et écraser mutuellement leurs résultats.



Problèmes de visibilité

En plus des conditions de course, les threads peuvent ne pas voir immédiatement les modifications effectuées par d'autres threads. Ceci est dû aux optimisations du processeur, du noyaux et de la JVM qui utilisent des caches locaux ou peuvent préempté la tache et/ou thread.

```java
public class ProblemeVisibilite {
    private static boolean arret = false;

    public static void main(String[] args) throws InterruptedException {
        Thread travailleur = new Thread(() -> {
            while (!arret) {
                // Travail...
            }
            System.out.println("Thread arrêté");
        });

        travailleur.start();
        Thread.sleep(2000);

        arret = true; // Peut ne pas être visible immédiatement !
        System.out.println("Demande d'arrêt envoyée");
    }
}
```

Section critique

> Une section critique est une portion de code où un ou plusieurs threads accèdent à des ressources partagées qui ne peuvent pas être utilisées simultanément de manière sûre. Cette définition, bien qu'apparemment simple, cache une complexité considérable dans sa mise en œuvre pratique.

Solutions de synchronisation

Pour résoudre ces problèmes, Java propose plusieurs mécanismes de synchronisation, chacun adapté à des situations spécifiques. Ces méchanismes, assuré par des instruction matériel et système permettent de géré les sections critiques à l'image des chemin de fer.

La métaphore du jeton ferroviaire (railway token) trouve ses origines dans l'histoire réelle des chemins de fer du XIXe siècle. Sur les lignes ferroviaires à voie unique, un système physique empêchait les collisions frontales : un jeton métallique unique était requis pour qu'un train puisse emprunter la section critique.

- Exclusion Mutuelle : Un seul train peut détenir le jeton à un moment donné, garantissant qu'aucune collision ne peut survenir sur la voie unique.
- Attente Obligatoire : Tout train désirant emprunter la voie doit d'abord obtenir le jeton. S'il n'est pas disponible, le train doit attendre en gare.

- Libération Obligatoire : À l'arrivée en gare de destination, le train doit obligatoirement rendre le jeton pour permettre à d'autres trains de circuler.
- Non-Préemption : Une fois qu'un train détient le jeton et circule sur la voie, il ne peut pas être forcé à s'arrêter ou à céder le passage.

Mutex avec synchronized

Le **mutex** (exclusion mutuelle) garantit qu'un seul thread peut exécuter une section critique à la fois. En Java, le mot-clé `synchronized` implémente ce mécanisme.

```
class CompteurSecurise {
    private int count = 0;

    // Méthode synchronisée
    public synchronized void incrementer() {
        count++; // Section critique protégée
    }

    public synchronized int getCount() {
        return count;
    }
}

class CompteurAvecBloc {
    private int count = 0;
    private final Object verrou = new Object();

    public void incrementer() {
        synchronized(verrou) {
            count++; // Seul ce bloc est synchronisé
        }
    }

    public int getCount() {
        synchronized(verrou) {
            return count;
        }
    }
}
```

Ressources





Regarder sur

Sémaphore

Le **sémaphore** contrôle le nombre de threads pouvant accéder simultanément à une ressource. Contrairement au mutex (qui permet 1 seul accès), le sémaphore peut autoriser N accès simultanés.

```java
import java.util.concurrent.Semaphore;

class PoolConnexions {
    private final Semaphore semaphore;
    private final List<Connexion> connexions;

    public PoolConnexions(int nbConnexions) {
        semaphore = new Semaphore(nbConnexions);
        connexions = new ArrayList<>();

        // Initialiser le pool
        for (int i = 0; i < nbConnexions; i++) {
            connexions.add(new Connexion("DB-" + i));
        }
    }

    public Connexion obtenirConnexion() throws InterruptedException {
        semaphore.acquire(); // Attendre qu'une place se libère

        synchronized(connexions) {
            return connexions.remove(connexions.size() - 1);
        }
    }

    public void libererConnexion(Connexion cnx) {
        synchronized(connexions) {
            connexions.add(cnx);
        }
        semaphore.release(); // Libérer une place
    }
}

// Utilisation
public class ExempleSemaphore {
    public static void main(String[] args) {
        PoolConnexions pool = new PoolConnexions(3); // 3 connexions max

        for (int i = 0; i < 10; i++) {
            new Thread(() -> {
                try {
                    Connexion cnx = pool.obtenirConnexion();
                    System.out.println("Connexion obtenue : " + cnx.getNom());

                    Thread.sleep(2000); // Simuler du travail

                    pool.libererConnexion(cnx);
                    System.out.println("Connexion libérée : " + cnx.getNom());
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }).start();
        }
    }
}
```

Read-Write Lock

Le **verrou lecteur-écrivain** optimise les accès en permettant plusieurs lecteurs simultanés, mais un seul écrivain à la fois.

```java
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;

class CacheSecurise<K, V> {
    private final Map<K, V> cache = new HashMap<>();
    private final ReadWriteLock rwLock = new ReentrantReadWriteLock();

    public V lire(K cle) {
        rwLock.readLock().lock(); // Plusieurs lecteurs OK
        try {
            return cache.get(cle);
        } finally {
            rwLock.readLock().unlock();
        }
```

```
    }

    public void ecrire(K cle, V valeur) {
        rwLock.writeLock().lock(); // Un seul écrivain
        try {
            cache.put(cle, valeur);
        } finally {
            rwLock.writeLock().unlock();
        }
    }

    public void supprimer(K cle) {
        rwLock.writeLock().lock();
        try {
            cache.remove(cle);
        } finally {
            rwLock.writeLock().unlock();
        }
    }
}
```

Spin Lock

Le **spin lock** est un verrou où les threads **`tournent en boucle`** au lieu de se bloquer, utile pour des sections critiques très courtes. On parle aussi d'attente active, elles consomme de la ressource CPU contrairement au blockage dont le temps peu être utilisé par un autre processus / thread.

```
import java.util.concurrent.atomic.AtomicBoolean;

class SpinLock {
    private final AtomicBoolean verrou = new AtomicBoolean(false);

    public void acquerir() {
        while (verrou.compareAndSet(false, true) == false) {
            // Tourner en boucle jusqu'à obtenir le verrou
            Thread.yield(); // Suggère au scheduler de changer de thread
        }
    }

    public void liberer() {
        verrou.set(false);
    }
}

// Utilisation
class CompteurSpinLock {
    private int count = 0;
    private final SpinLock spinLock = new SpinLock();

    public void incrementer() {
        spinLock.acquerir();
        try {
            count++; // Section critique très courte
        } finally {
            spinLock.liberer();
        }
    }

    public int getCount() {
        spinLock.acquerir();
        try {
            return count;
        } finally {
            spinLock.liberer();
        }
    }
}
```

Quand utiliser un spin lock ?
- Sections critiques très courtes (quelques instructions)
- Faible contention (peu de threads en compétition)
- Systèmes multiprocesseurs (évite les changements de contexte coûteux)

Attention : Les spin locks consomment continuellement du CPU même en attente. Ils sont inadaptés pour des sections critiques longues ou sur des systèmes monoprocesseur.

Deadlock

Un **deadlock** (interblocage) survient lorsque plusieurs threads s'attendent mutuellement, créant une situation d'blocage permanent.

```
public class ExempleDeadlock {
    private static final Object verrou1 = new Object();
    private static final Object verrou2 = new Object();

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            synchronized (verrou1) {
                System.out.println("Thread 1 : verrou1 acquis");

                try { Thread.sleep(100); } catch (InterruptedException e) {}

                System.out.println("Thread 1 : attente verrou2...");
                synchronized (verrou2) {
                    System.out.println("Thread 1 : verrou2 acquis");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (verrou2) {
                System.out.println("Thread 2 : verrou2 acquis");

                try { Thread.sleep(100); } catch (InterruptedException e) {}

                System.out.println("Thread 2 : attente verrou1...");
                synchronized (verrou1) {
                    System.out.println("Thread 2 : verrou1 acquis");
                }
            }
        });

        t1.start();
        t2.start();
        // Programme bloqué indéfiniment !
    }
}
```

Prévention du deadlock :

1. **Ordre d'acquisition fixe** : toujours acquérir les verrous dans le même ordre
2. **Timeout** : abandonner après un délai d'attente
3. **Éviter les verrous imbriqués** quand possible

Conclusion

Les threads permettent la programmation concurrente efficace en partageant les ressources d'un processus, mais introduisent des défis de synchronisation complexes. Le choix du mécanisme approprié dépend du contexte :

- **Mutex/synchronized** : pour l'exclusion mutuelle simple
- **Sémaphore** : pour contrôler le nombre d'accès simultanés
- **Read-Write Lock** : pour optimiser les accès lecture/écriture
- **Spin Lock** : pour les sections critiques très courtes

La maîtrise de ces concepts est essentielle pour développer des applications multithreadées robustes et performantes.

© Jehan-Antoine Vayssade 2011-2025